

Desarrollo por Contratos (DbC)

Construyendo Software Confiable y Verificable.

[Course Name] SW707 Construcción de Software II – Semana 3 | Universidad Nacional de Ingeniería (UNI)

Gemini Notebook

El Problema de la Confianza Ciega

Enlace dinámico y complejidad orientada a objetos (Semana 2).
Cuando un módulo llama a otro…



Si falla el input:
¿Debió validarlo el Módulo A antes de enviarlo?

Si falla el output:
¿Debió garantizarlo el Módulo B tras procesarlo?

El Resultado sin reglas:
Suposiciones ocultas y fallos imprevistos en tiempo de ejecución.

Gemini Notebook

El Enfoque Design by Contract (DbC)

Sin suposiciones ocultas. Solo acuerdos formales.



CLIENTE (Módulo A)

- Obligación: Proveer datos de entrada válidos y en estado correcto.
- Derecho: Recibir un resultado garantizado sin tener que verificarlo de nuevo.

PROVEEDOR (Módulo B)

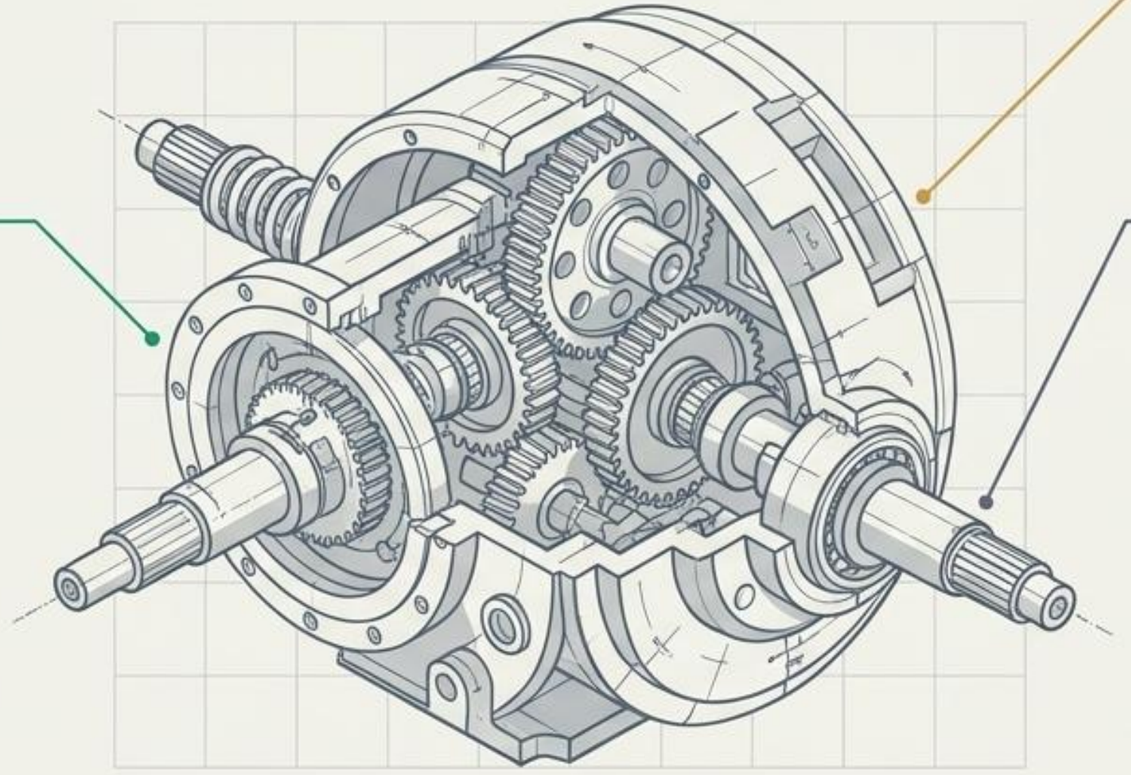
- Obligación: Computar el resultado exacto prometido y mantener la integridad del sistema.
- Derecho: Asumir incondicionalmente que los datos de entrada son correctos (no necesita validar internamente).

Gemini Notebook

La Trinidad del Desarrollo por Contratos

3. Invariantes de Clase (El Escudo Protector)

Envuelve toda la estructura. Reglas de integridad que deben ser ciertas antes y después de cualquier operación pública.



1. Precondiciones (La Puerta de Entrada)

Se evalúa antes de que la rutina comience. Filtra lo que tiene permitido ingresar.

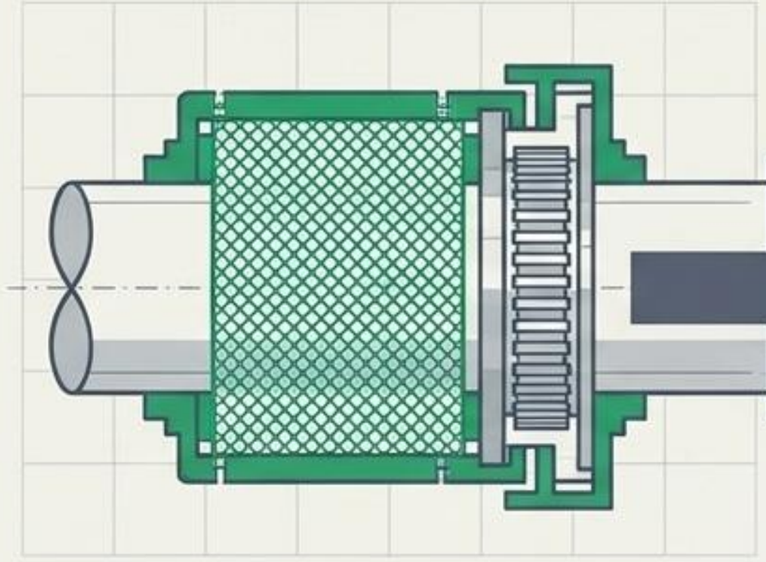
2. Postcondiciones (El Sello de Salida)

Se evalúa después de que la rutina termina. Garantiza lo que se entrega al exterior.

Gemini Notebook

Anatomía de una Rutina Contractual

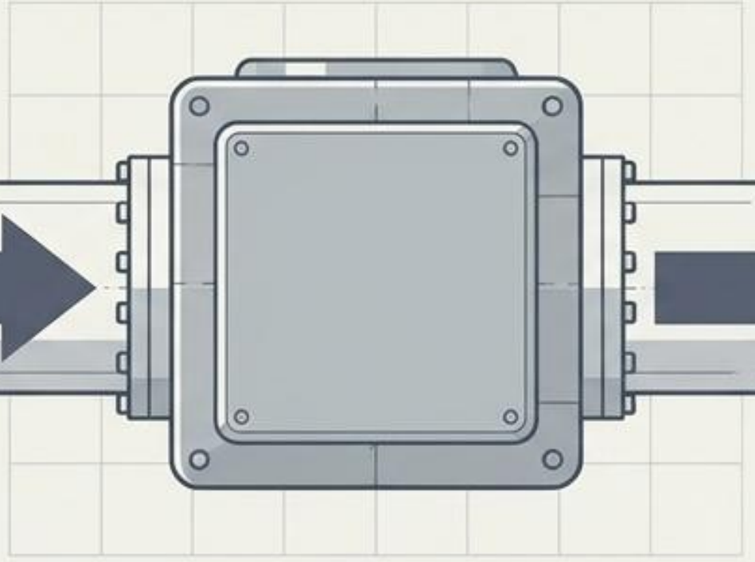
Filtro de Entrada
(Precondición)



Obligación del Cliente.

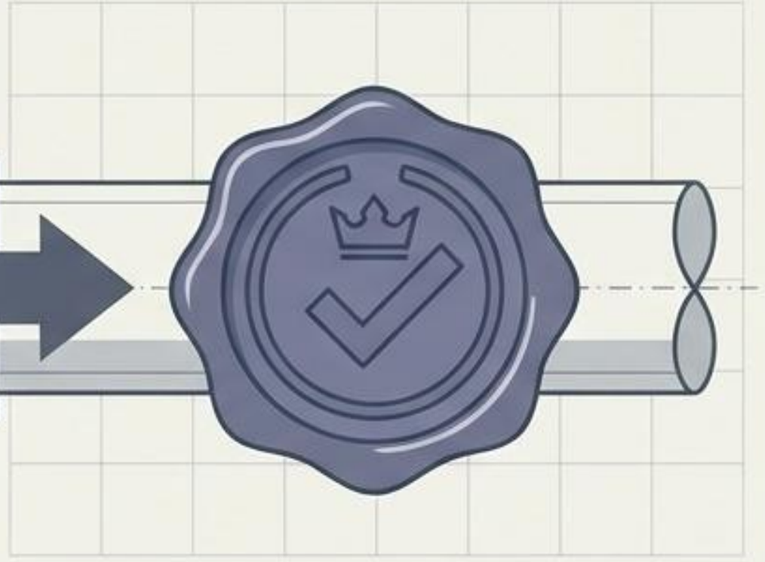
```
require: saldo >= retiro
    AND retiro > 0
```

Caja Negra
(La Rutina)



Lógica de negocio pura, libre de condicionales defensivos redundantes.

Sello de Salida
(Postcondición)



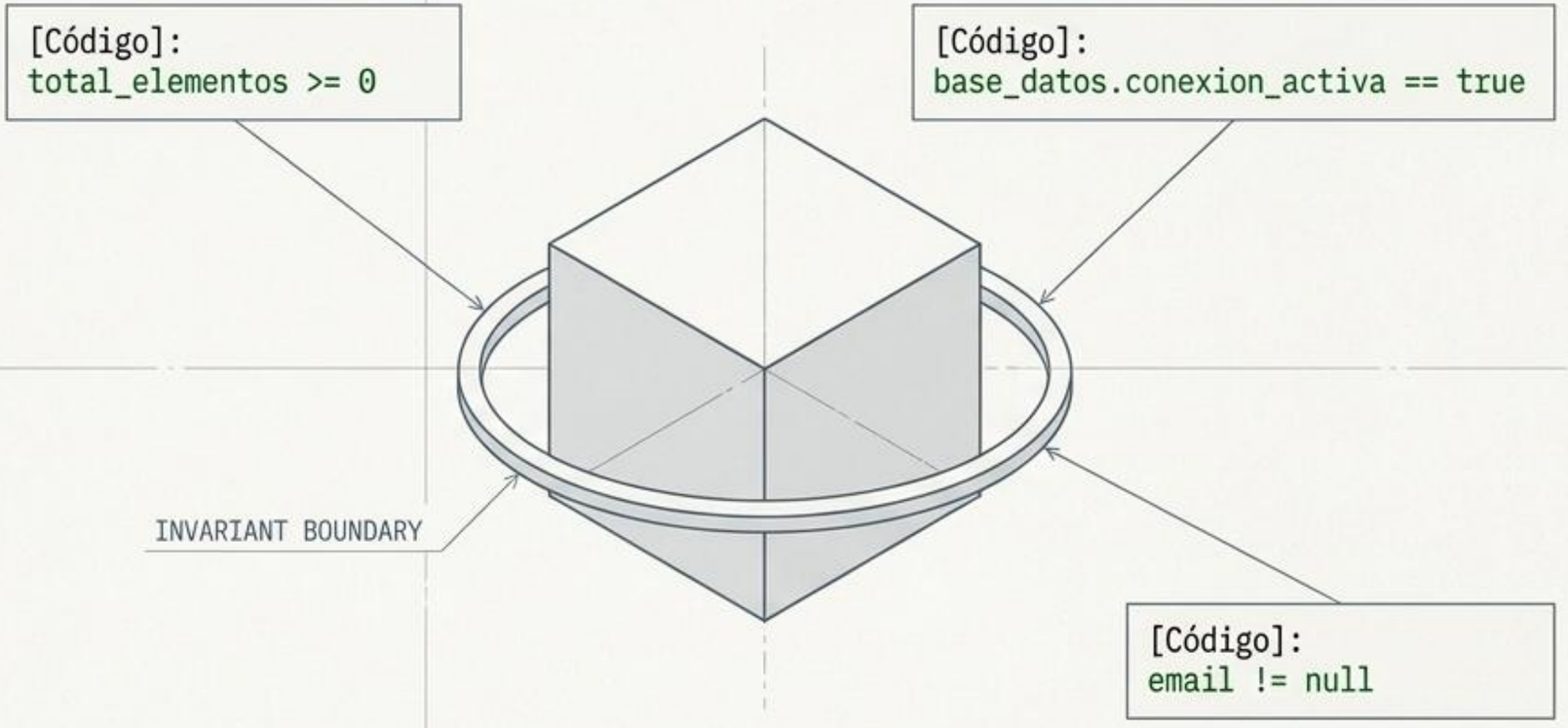
Obligación del Proveedor.

```
ensure: saldo ==
   old saldo - retiro
```

Gemini Notebook

El Invariante: Integridad a Través del Tiempo

Mientras que pre y postcondiciones aplican a rutinas individuales, el invariante es la “constitución” de la clase.
Es el fundamento del modelado de estado correcto.



Regla de Oro: Un método público puede romper el invariante momentáneamente durante su ejecución interna, pero debe restaurarlo antes de retornar el control al cliente.

Gemini Notebook

Matriz de Diagnóstico DbC

Mapeo de responsabilidades y fallos en tiempo de ejecución

Tipo de Contrato	Momento de Evaluación	¿Quién es Responsable?	Si Falla, el Bug está en...
`Precondición`	Antes de la rutina	El Cliente (Llamador)	El código del Cliente
`Postcondición`	Después de la rutina	El Proveedor (Invocado)	El código del Proveedor
`Invariante`	Antes y después de cualquier método público	El Proveedor (Diseño de Clase)	La arquitectura de estado del Proveedor

Gemini Notebook

De la Teoría a la Práctica: Aserciones Ejecutables

El Contrato Pasivo

Documentación Tradicional (Papel)

```
// PRE: amount must be > 0
```

Falla en silencio si se ignora.
Confianza ciega.



Verificación en Tiempo de Ejecución

El Contrato Activo

Aserciones Ejecutables (Con "Dientes")

```
assert(amount > 0, "Amount
        must be positive");
```

Validación mecánica en runtime.
Estrategia "Fail-Fast" (abortar rápidamente si el contrato se rompe).

Gemini Notebook

Principio de Separación Comando-Consulta (CQS)

Preguntar no debe cambiar la respuesta.



Consultas / Queries

Retornan un resultado.
NO modifican el estado observable del sistema.
(Son **inmutables** / seguras para usar en aserciones).

Comandos / Commands



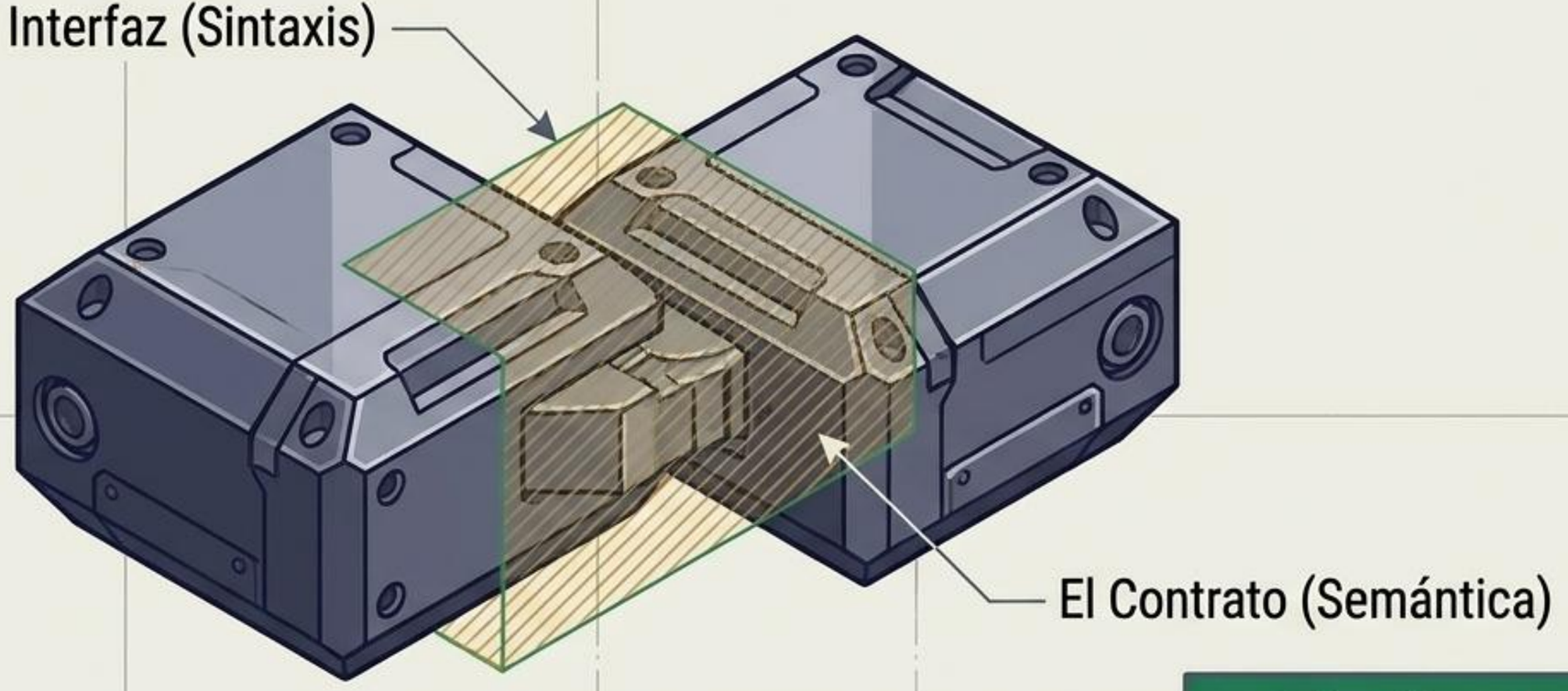
Modifican el estado del sistema.
NO retornan datos.



¿Por qué es vital para DbC? Evaluar una precondición o invariante (una consulta) jamás debe tener efectos secundarios impredecibles.

Gemini Notebook

Contratos en Interfaces y Reúso Confiable



La Limitación de las Interfaces Tradicionales

En la Orientación a Objetos estándar, una interfaz solo garantiza firmas y tipos (sintaxis). El compilador aprueba, pero el comportamiento dinámico es un misterio.

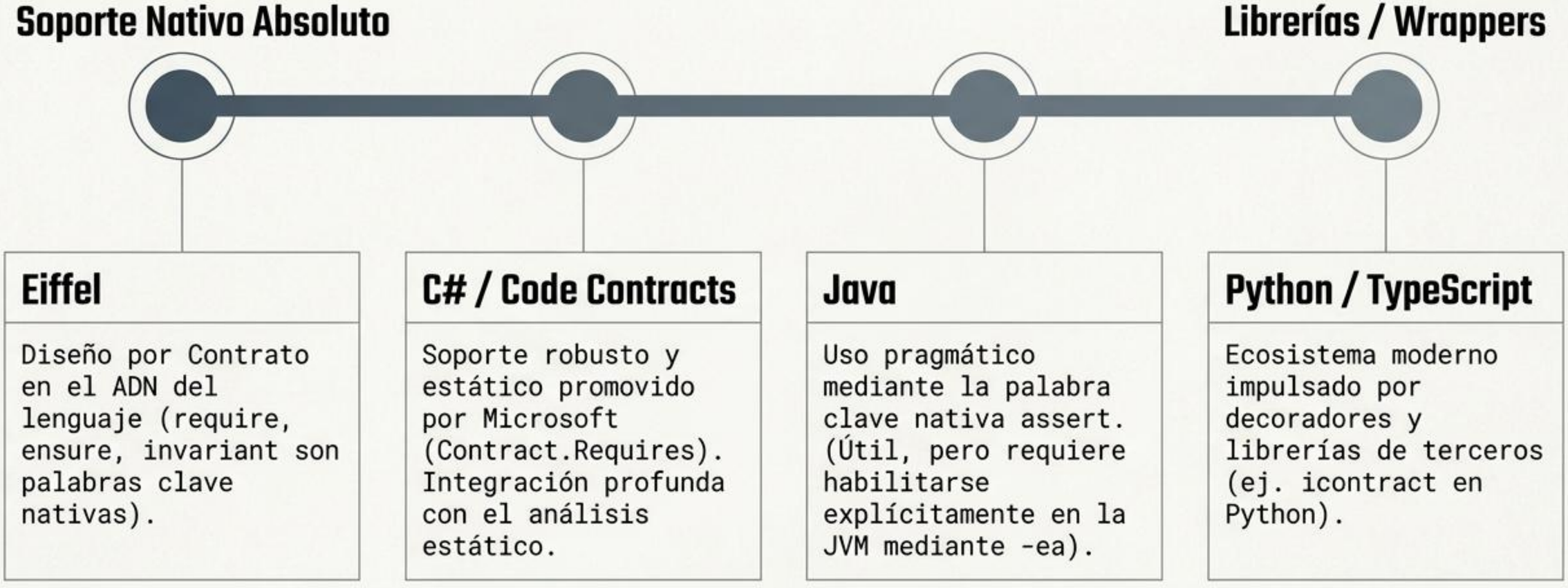
Interfaces con Contratos

Al vincular aserciones a la interfaz, garantizamos el comportamiento (semántica).

Polimorfismo Seguro:
Cualquier clase concreta que implemente la interfaz debe honrar el contrato original. Fundamental para una cadena de suministro robusta.

Gemini Notebook

Paisaje de Herramientas y Soporte en el Mundo Real



Gemini Notebook

¿Contratos o Barricadas?

Preparando el terreno para la Semana 4 (Programación Defensiva)



DbC / Contratos

- **Premisa:** Asume Confianza.
- **Ubicación:** Entre módulos internos del mismo sistema bajo nuestro control.
- **Filosofía:** "Si me das basura, el sistema aborta inmediatamente (Bug tuyo)."

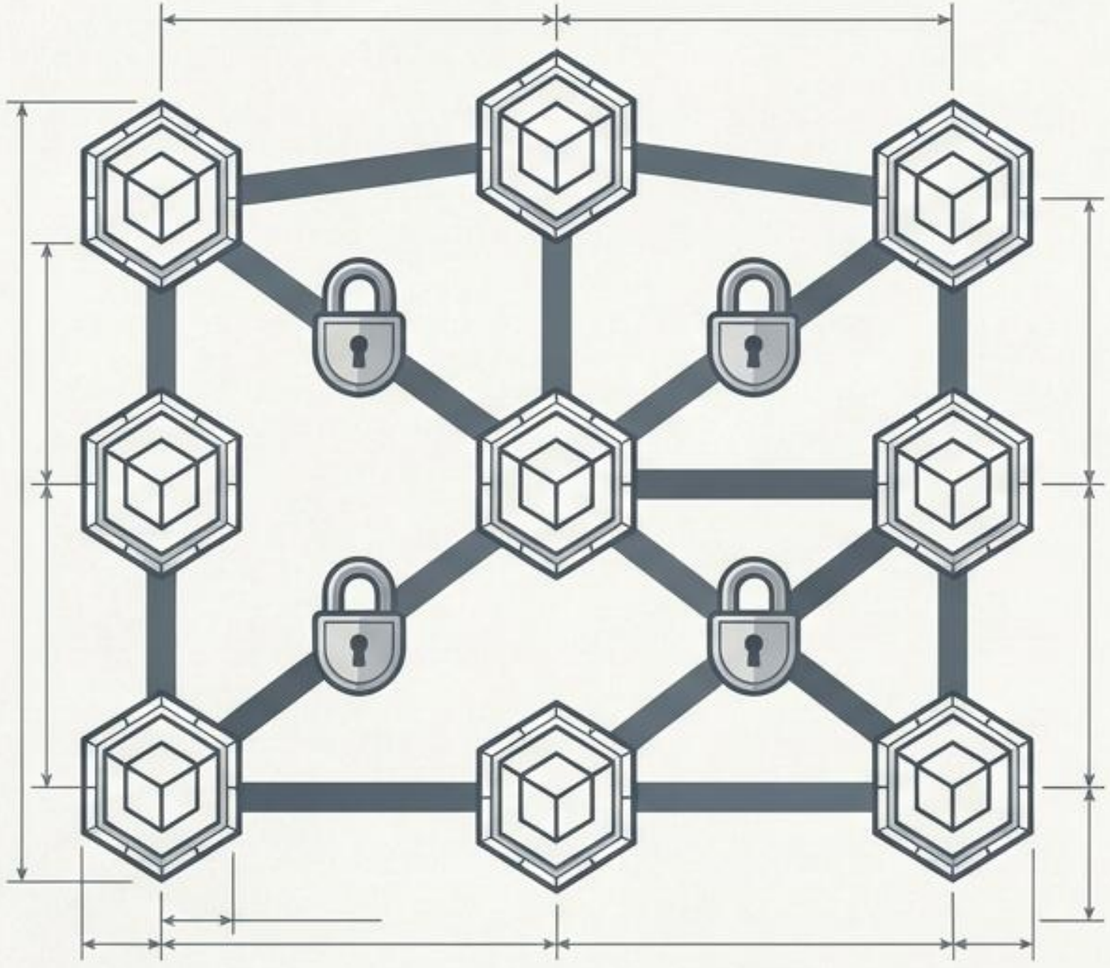


Programación Defensiva

- **Premisa:** Asume Hostilidad.
- **Ubicación:** En las fronteras externas del sistema (APIs públicas, inputs de usuario).
- **Filosofía:** "Si me das basura, la sanitizo, la valido y manejo el error gracefully."

Gemini Notebook

Construcción para la Verificación



Diseño Detallado

DbC no es solo para atrapar bugs en *runtime*. Es una herramienta de diseño arquitectónico que fuerza a pensar en responsabilidades antes de codificar.

Documentación Viva

A diferencia de los manuales que se desactualizan, los contratos ejecutables son la única documentación que fuerza al código a decir la verdad.

Aislamiento de Fallos

Cuando un sistema complejo falla, el contrato roto señala exactamente qué módulo rompió su promesa, reduciendo el tiempo de debugging a minutos.

Gemini Notebook

Del Código que 'Funciona' al Código que 'Garantiza'

El cambio de mentalidad del Ingeniero de Software



Piensa en Obligaciones, no en Algoritmos.

Antes de escribir un if o un bucle, define el acuerdo legal de la rutina.



Falla Rápido, Falla Fuerte.

Usa aserciones sin miedo. Un sistema que aborta a tiempo es infinitamente más seguro que uno que propaga estado corrupto en silencio.



Tu Turno (Laboratorio).

Próximas 4 horas: Implementación de componentes con aserciones rigurosas y contratos explícitos. Preparen sus entornos.

Gemini Notebook